

MITM Secure Communication System

Test Scenarios & Test Cases Document

Project: Python-based Secure Communication with Passive & Active MITM Attack Simulation

Scenario 1: Secure Direct Communication (No Attacker)

Validates the baseline hybrid RSA+AES encrypted communication between Sender (A) and Receiver (B) without any intermediate attacker.

Covers both text and audio message types.

S. No	Test Scenario	Pre Condition	Post Condition	Priority	Comments
1	Verify secure text message transmission between Sender and Receiver using hybrid RSA+AES encryption	- receiver.py running on port 9001 - RSA keys generated in keys/ - receiver_public.pem copied to Sender - pycryptodome installed on both	- Text message decrypted correctly on Receiver - SHA-256 integrity check passed - No errors in logs	High	Baseline test — must pass before running MITM scenarios
2	Verify secure audio message transmission between Sender and Receiver using hybrid RSA+AES encryption	- receiver.py running on port 9001 - RSA keys generated in keys/ - sounddevice or numpy available - receiver_public.pem copied to Sender	- Audio PCM bytes decrypted correctly on Receiver - Playback produces original audio - SHA-256 integrity check passed	High	Audio baseline — confirms binary payload handling
3	Verify RSA key generation and persistence across multiple receiver restarts	- receiver.py not yet started - keys/ directory empty or absent - pycryptodome installed	- Keys generated on first run - Same keys loaded on subsequent runs - No duplicate key generation	Medium	Key management stability test

Test Cases — Scenario 1

S.No	Test Case ID	Test Case Description	Precondition	Test Data	Expected Result	Postcondition	Status	Comments
1	TC-SC1-001	Send a short text message and verify correct decryption on Receiver	- receiver.py running - Public key on Sender - TCP connection established	Message: 'Hello Secure World'	Receiver prints: Text message received: Hello Secure World	Message logged, connection stays open	Pass	Null
2	TC-SC1-002	Send a long text message (>500 chars) and verify complete decryption without truncation	- receiver.py running - Public key on Sender	Message: 500-char random alphanumeric string	Full message received and displayed intact with correct SHA-256 digest	No truncation errors in logs	Pass	Null
3	TC-SC1-003	Send text message containing special characters and Arabic Unicode	- receiver.py running - Public key on Sender	Message: 'Test مرحبا @\$% 123'	Receiver prints exact message including Arabic and special chars	UTF-8 encoding handled correctly	Pass	Null
4	TC-SC1-004	Send a simulated audio message (numpy noise) and verify correct byte-count on Receiver	- receiver.py running - sounddevice not installed (fallback mode)	Audio: 3-second synthetic noise (44100 Hz, int16, mono) = 176400 bytes	Receiver prints: Audio message received (176400 bytes PCM)	Byte count matches original PCM size	Pass	Null
5	TC-SC1-005	Verify SHA-256 digest matches original plaintext after decryption	- receiver.py running - crypto_utils.py loaded	Any text message	hybrid_decrypt returns plaintext without ValueError — integrity verified	No INTEGRITY FAILURE in logs	Pass	Null
6	TC-SC1-006	Send multiple consecutive messages in one session and verify all are received correctly	- receiver.py running - Sender connected	3 consecutive messages: 'Message 1', 'Message 2', 'Message 3'	All 3 messages received in order with correct content	Session remains open, no disconnection	Pass	Null
7	TC-SC1-007	Verify RSA key persistence — restart receiver and confirm existing keys are reloaded	- keys/ directory exists with valid PEM files - receiver.py stopped	Restart receiver.py	Log shows: 'Loading existing RSA key-pair' — no new key generated	Sender can still communicate using old public key	Pass	Null
8	TC-SC1-008	Attempt to connect sender with wrong/missing public key and verify failure	- receiver.py running - keys/ directory empty on Sender	Run sender.py with no public key	sender.py logs error and exits: 'Receiver public key not found'	No packet sent, clean exit	Pass	Null

Scenario 2: Passive MITM Attack

Validates attacker_passive.py acting as a transparent TCP proxy. Confirms that all packets are intercepted and logged, that the attacker cannot read plaintext, and that the receiver still gets the original message unmodified. Covers text and audio payloads.

S. No	Test Scenario	Pre Condition	Post Condition	Priority	Comments
1	Verify passive proxy intercepts text message and forwards it transparently to Receiver without modification	- receiver.py running on port 9001 - attacker_passive.py running on port 9000 - Sender directed to port 9000 - receiver_public.pem on Sender only	- Intercepted ciphertext logged in intercept.log - Receiver gets original plaintext - Attacker cannot read content	High	Core passive scenario — attacker sees encrypted bytes only
2	Verify passive proxy intercepts audio message, logs it, and forwards to Receiver without corruption	- receiver.py running on port 9001 - attacker_passive.py running on port 9000 - Sender uses option [2] for audio	- Audio ciphertext logged (large blob) - Receiver plays back original audio correctly - Attacker log shows type=audio	High	Audio interception — attacker gets noise if ciphertext played as PCM
3	Verify intercept.log file is created and contains all required fields for each intercepted packet	- attacker_passive.py running - At least one message sent through proxy	- intercept.log exists - Each entry has: timestamp, direction, msg_type, ciphertext_b64, encrypted_key_b64, digest_hex	Medium	Log completeness validation

Test Cases — Scenario 2

S.No	Test Case ID	Test Case Description	Precondition	Test Data	Expected Result	Postcondition	Status	Comments
1	TC-PS2-001	Verify proxy receives Sender's connection on port 9000 and establishes connection to Receiver on port 9001	- receiver.py running on 9001 - attacker_passive.py started on 9000	sender.py connecting to 192.168.1.30:9000	Proxy logs: 'Sender connected from X:Y — proxying to 192.168.1.20:9001'	Three-party session established	Pass	Null
2	TC-PS2-002	Verify intercepted text packet is logged with correct fields in intercept.log	- Full passive setup running - Sender sent text message	Message: 'secure test message'	intercept.log entry contains: - msg_type: text - ciphertext_b64: non-empty - encrypted_key_b64: 256 bytes encoded - digest_hex: 64-char hex string	Log entry written and readable	Pass	Null
3	TC-PS2-003	Verify attacker cannot reconstruct plaintext from intercepted ciphertext without private key	- intercept.log contains a captured packet - Attacker has NO private key	Attempt AES decrypt with random key on captured ciphertext	Decryption raises ValueError or produces garbage output — original text unrecoverable	Attacker remains unable to read content	Pass	Null
4	TC-PS2-004	Verify Receiver gets original unmodified text after passing through passive proxy	- Full passive setup running	Message: 'hello from sender'	Receiver displays: Text message received: hello from sender (identical to sent text)	Integrity check passed on Receiver	Pass	Null
5	TC-PS2-005	Verify audio ciphertext is logged and Receiver plays back original audio after passive interception	- Full passive setup running - sounddevice or numpy available	Audio: 3-second synthetic noise (176400 bytes PCM)	- Attacker log: type=audio, ciphertext_len ~176656 bytes - Receiver: plays original audio without distortion	Audio integrity preserved end-to-end	Pass	Null
6	TC-PS2-006	Verify intercepted audio ciphertext produces noise/garbage when played directly as raw PCM	- intercept.log has captured audio ciphertext_b64 - numpy/sounddevice available	Decode base64 ciphertext, play as int16 PCM at 44100 Hz	Output is random noise — no recognizable speech or audio content	Confirms encryption effectiveness on binary payloads	Pass	Null
7	TC-PS2-007	Verify proxy handles multiple sequential messages without dropping packets	- Full passive setup running - Sender in active session	Send 5 consecutive text messages	All 5 messages received by Receiver in correct order All 5 entries appear in intercept.log	No packet loss through proxy	Pass	Null
8	TC-PS2-008	Verify proxy gracefully handles Sender disconnection	- Full passive setup running - Sender connected	Sender sends 'q' to quit	Proxy logs: 'Sender X:Y disconnected' Proxy continues listening for new connections	No crash, proxy stays alive	Pass	Null

Scenario 3: Active MITM Attack

Validates attacker_active.py which intercepts, decrypts, tampers with, re-encrypts, and forwards modified packets. Covers text substitution, audio corruption, and integrity detection using SHA-256 digest comparison.

S. No	Test Scenario	Pre Condition	Post Condition	Priority	Comments
1	Verify active proxy intercepts text message, modifies content, re-encrypts, and delivers tampered message to Receiver	- receiver.py running on port 9001 - attacker_active.py running on port 9000 with --mode text - Attacker has BOTH RSA keys (private + public) - Sender directed to port 9000	- Receiver gets tampered text (**REDACTED** and TAMPERED tag appended) - Attacker console shows Original vs Modified comparison	High	Core active text attack — demonstrates content manipulation
2	Verify active proxy corrupts audio payload via XOR and delivers distorted audio to Receiver	- receiver.py running on port 9001 - attacker_active.py running with --mode audio - Attacker has both RSA keys - Sender sends audio message	- Receiver plays distorted noise instead of original audio - Attacker console shows 'Modified audio: X bytes corrupted'	High	Active audio attack — XOR corruption of PCM samples
3	Verify integrity failure detection when Receiver compares tampered digest against known original digest from out-of-band channel	- receiver.py running - active attack in progress - Receiver has pre-stored SHA-256 digest of expected message	- Receiver logs: INTEGRITY FAILURE - Message discarded without display - No crash on Receiver	High	Defense validation — confirms SHA-256 detects tampering

Test Cases — Scenario 3

S.No	Test Case ID	Test Case Description	Precondition	Test Data	Expected Result	Postcondition	Status	Comments
1	TC-AM3-001	Verify attacker successfully decrypts intercepted text using stolen private key	- attacker_active.py started with both keys - Sender sends text through proxy	Message: 'my password is secret123'	Attacker console shows: Decrypted plaintext (24 bytes): b'my password is secret123'	Attacker has full plaintext access	Pass	Null
2	TC-AM3-002	Verify 'password' keyword is replaced with ***REDACTED*** in tampered text	- Active proxy running --mode text - Sender sends message containing 'password'	Message: 'my password is secret123'	Receiver gets: 'my ***REDACTED*** is secret123 [TAMPERED BY ATTACKER]'	Substitution applied correctly	Pass	Null
3	TC-AM3-003	Verify [TAMPERED BY ATTACKER] tag is appended to all intercepted text messages	- Active proxy running --mode text	Message: 'hello world' (no 'password' keyword)	Receiver gets: 'hello world [TAMPERED BY ATTACKER]'	Tag always appended regardless of content	Pass	Null
4	TC-AM3-004	Verify tampered message is re-encrypted with receiver's public key and Receiver can decrypt it	- Active proxy running --mode text - Receiver has private key	Any text message	Receiver decrypts tampered packet without cryptographic errors — tampered text displayed	Re-encryption transparent to Receiver's crypto layer	Pass	Null
5	TC-AM3-005	Verify audio XOR corruption: every other int16 sample is bit-flipped	- Active proxy running --mode audio - Audio PCM bytes known	Audio: 176400 bytes int16 PCM	First 4 bytes of original: [AB CD EF 12] First 4 bytes of modified: [54 32 EF 12] (every 4-byte stride: first 2 XOR 0xFF)	XOR pattern applied at correct stride	Pass	Null
6	TC-AM3-006	Verify Receiver plays distorted/noisy audio after active audio attack	- Active proxy running --mode audio - sounddevice available on Receiver	Send 3-second voice audio	Receiver plays unrecognizable distorted noise — no original speech audible	Audio corruption confirmed on playback	Pass	Null
7	TC-AM3-007	Verify --mode both applies text tampering to text messages and audio corruption to audio messages in same session	- Active proxy running --mode both	Send 1 text + 1 audio message in same session	Text message: tampered with REDACTED tag Audio message: distorted noise on Receiver	Mode 'both' correctly routes tampering by msg_type	Pass	Null
8	TC-AM3-008	Verify integrity failure is raised when Receiver checks tampered digest against a pre-known correct digest	- Active attack running - Receiver has correct SHA-256 of original message stored externally	Original digest: sha256('hello world') Tampered content delivered by attacker	hybrid_decrypt or post-check raises ValueError: 'Integrity check FAILED — message may have been tampered with'	Message discarded, INTEGRITY FAILURE logged	Pass	Null
9	TC-AM3-009	Verify active proxy handles audio message type correctly when --mode is set to text only (no audio tampering applied)	- Active proxy running --mode text	Send audio message through active proxy	Audio forwarded unmodified — Receiver plays original audio correctly Proxy log: 'mode does not match msg_type — pass through'	Mode filtering works correctly	Pass	Null
10	TC-AM3-010	Verify active proxy does not crash when Sender disconnects mid-session	- Active proxy running - Sender connected and mid-message	Kill sender process abruptly	Proxy logs: 'Sender→Receiver forwarder stopped: peer disconnected' Proxy continues listening for new connections	No unhandled exception, proxy stays alive	Pass	Null